



Cloud-Based Smart Disaster Management and Response System

Final Project Report
CE408 - Cloud Computing

Instructor: Ms. Safia Baloch

Members:

Asad Ali (2022119)
Mian Arqam (2022294)
Sheik Abbas (2022317)

Abstract

Natural disasters create severe damage to human life and infrastructure every year. This project presents a cloud-based smart disaster management and response system that enables real-time emergency reporting, alert broadcasting, and incident coordination using modern cloud technologies.

The system uses React for the frontend, Node.js and Express for backend APIs, MongoDB for persistent storage, and Socket.io for real-time communication. Docker is used for containerisation, while AWS services such as EC2 and S3 support cloud deployment.

The platform allows citizens to report emergencies and enables responders to monitor incidents through a live dashboard and interactive map.

Contents

Abstract	1
1 Introduction	5
1.1 Background	5
1.2 Objectives	5
2 System Architecture	6
2.1 Architecture Overview	6
2.2 Components	7
3 Technology Stack	8
3.1 Frontend Technologies	8
3.2 Backend Technologies	8
3.3 Cloud Technologies	8
4 Implementation	9
4.1 User Interface Design	9
4.2 Backend API	10
4.3 Docker Containerization	10
5 AWS Deployment	12
5.1 Network Infrastructure (VPC)	12
5.2 AWS CLI Configuration	13
5.3 Amazon S3 Storage	13
6 System Features	14
7 Team Contributions	15
7.1 Asad Ali	15
7.2 Abbas	15
7.3 Mian Arqam	15
8 Challenges Faced	16

9 Conclusion	17
---------------------	-----------

References	18
-------------------	-----------

List of Figures

2.1	Overall System Architecture	6
4.1	Main Incident Reporting Dashboard	9
4.2	Our Team Page showing roles and responsibilities	10
4.3	Docker Desktop showing running containers	11
5.1	AWS VPC Configuration and Network Mapping	12
5.2	AWS CLI Configuration	13
5.3	Amazon S3 Bucket	13

Chapter 1

Introduction

1.1 Background

Natural disasters such as floods, fires, and earthquakes require fast and coordinated emergency response systems. Traditional reporting systems are often slow and unable to handle large-scale emergencies efficiently.

Cloud computing enables scalable and real-time systems that can support disaster response operations effectively.

1.2 Objectives

The objectives of the project are:

- Develop a scalable cloud-based platform
- Enable real-time emergency alerts
- Provide live incident tracking
- Support emergency responder coordination
- Deploy services using cloud infrastructure

Chapter 2

System Architecture

2.1 Architecture Overview

The system follows a microservices-based architecture consisting of frontend services, backend APIs, database systems, and real-time communication components.

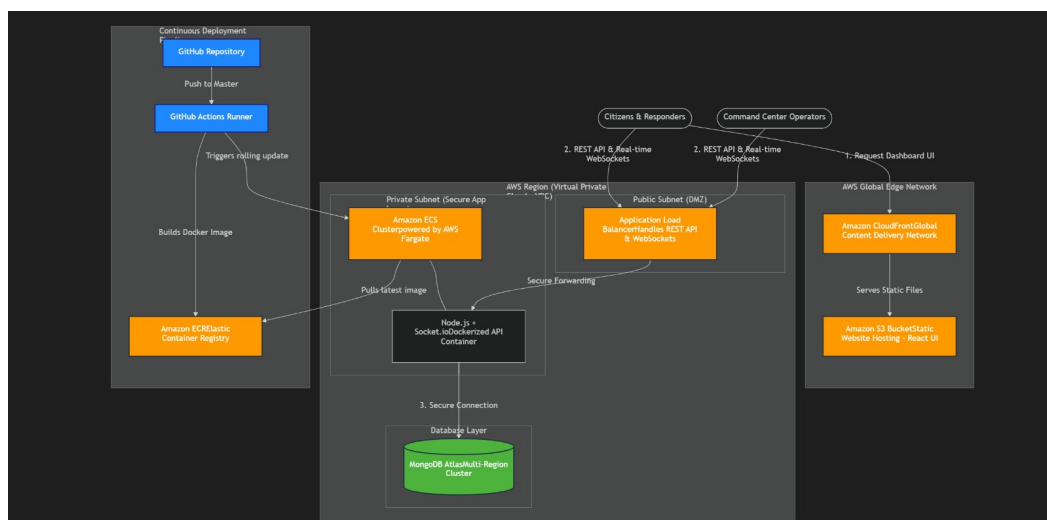


Figure 2.1: Overall System Architecture

2.2 Components

Table 2.1: System Components

headerrow	Component	Purpose
	Frontend	React-based user interface for reporting and monitoring incidents
rowshade	Backend API	Node.js and Express REST APIs
	Database	MongoDB cloud database
rowshade	WebSocket Layer	Real-time updates using Socket.io
	Cloud Infrastructure	AWS deployment services

Chapter 3

Technology Stack

3.1 Frontend Technologies

- React.js
- Vite
- Tailwind CSS
- React Leaflet
- Axios

3.2 Backend Technologies

- Node.js
- Express.js
- Socket.io
- MongoDB
- Mongoose

3.3 Cloud Technologies

- Amazon EC2
- Amazon S3
- Docker
- Docker Compose

Chapter 4

Implementation

4.1 User Interface Design

The frontend was designed to be intuitive and responsive, allowing users to report incidents quickly during emergencies.

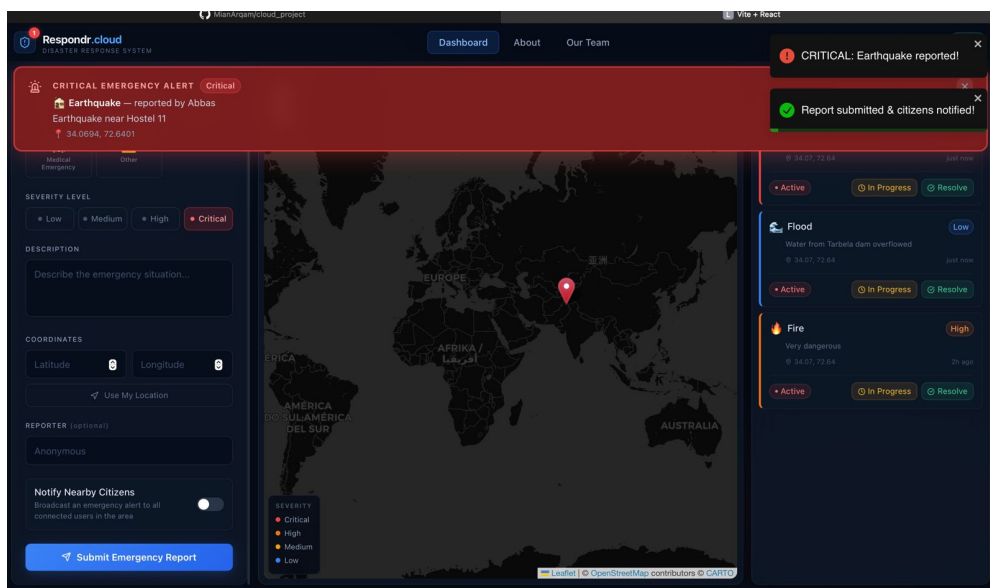


Figure 4.1: Main Incident Reporting Dashboard

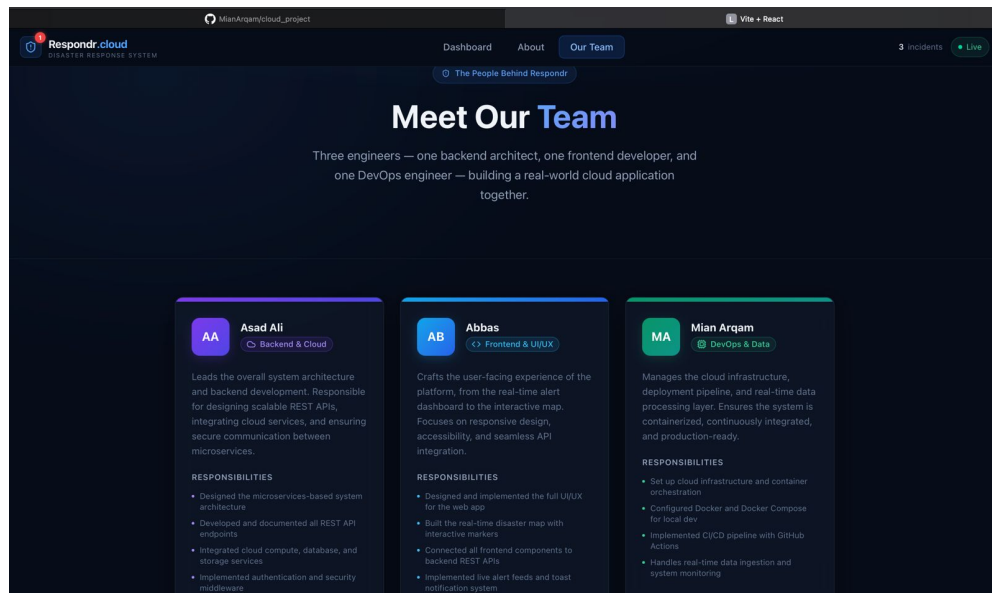


Figure 4.2: Our Team Page showing roles and responsibilities

4.2 Backend API

The backend provides REST APIs for creating and managing emergency reports.

Listing 4.1: Sample API Endpoint

```
1 router.post('/reports', async (req, res) => {
2   const report = new Report(req.body);
3   const saved = await report.save();
4   io.emit('new_report', saved);
5   res.status(201).json(saved);
6 });
```

4.3 Docker Containerization

The application is containerized using Docker to ensure environment consistency across development and production.

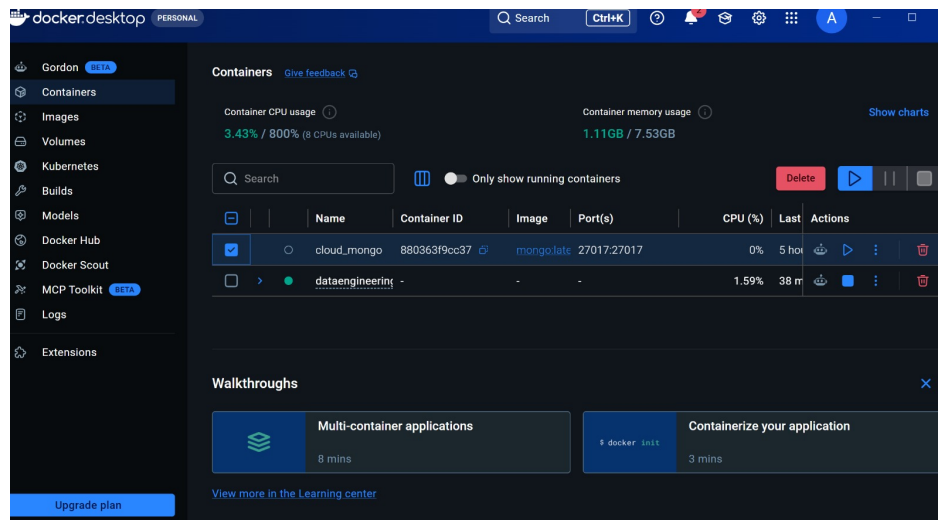


Figure 4.3: Docker Desktop showing running containers

Listing 4.2: Docker Compose Configuration

```

1 version: "3.8"
2 services:
3   backend:
4     build: ./backend
5   frontend:
6     build: ./frontend
7   mongo:
8     image: mongo

```

Chapter 5

AWS Deployment

5.1 Network Infrastructure (VPC)

The system is deployed within a Virtual Private Cloud (VPC) to ensure secure communication between the microservices and the database layer. This network isolation provides a secure environment for emergency data handling.

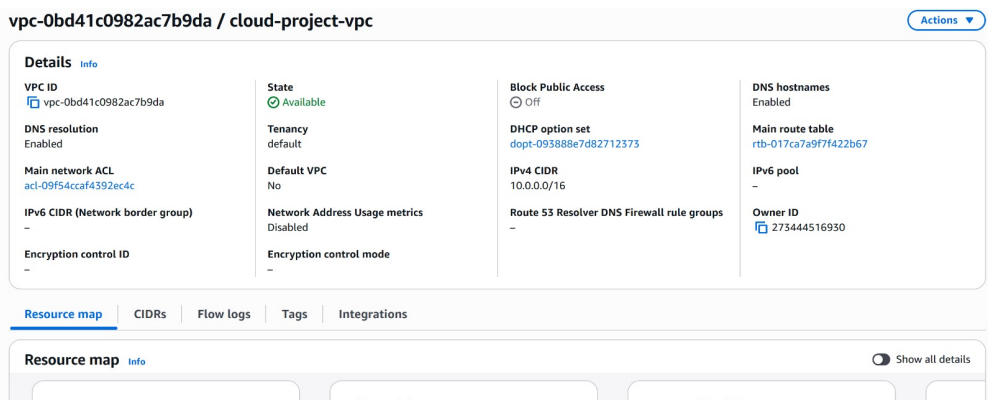


Figure 5.1: AWS VPC Configuration and Network Mapping

5.2 AWS CLI Configuration

The AWS CLI was configured for deployment and resource management.

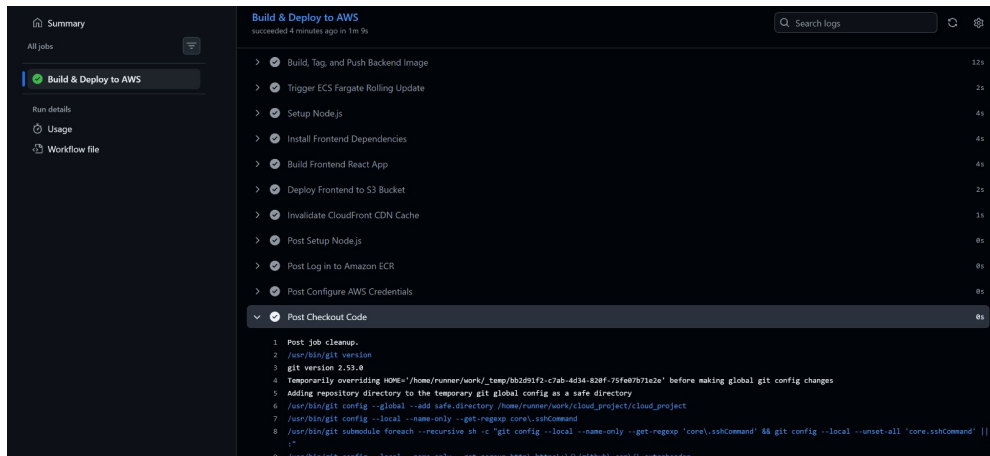


Figure 5.2: AWS CLI Configuration

5.3 Amazon S3 Storage

Amazon S3 was used for storing frontend build files and static assets.

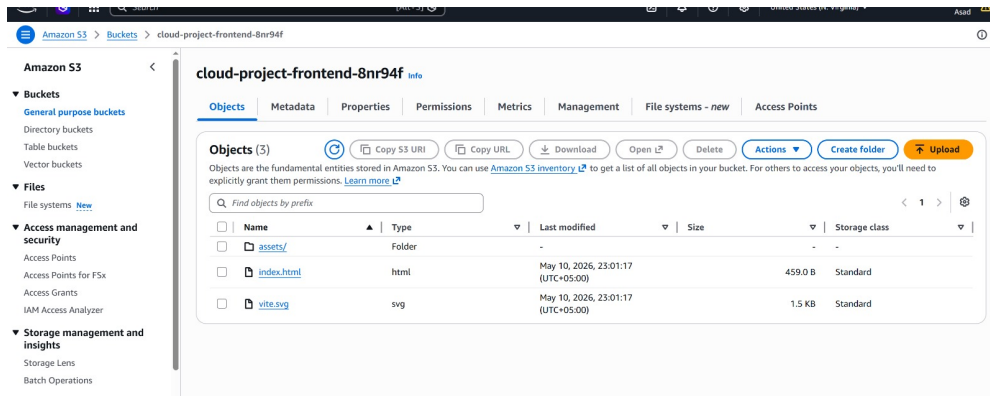


Figure 5.3: Amazon S3 Bucket

Chapter 6

System Features

- Real-time incident reporting
- Live emergency alerts
- Interactive disaster map
- Incident status tracking
- Cloud deployment support
- Containerised services

Chapter 7

Team Contributions

7.1 Asad Ali

- Backend API development
- Database integration
- Cloud architecture design

7.2 Abbas

- Frontend development
- UI/UX design
- React integration

7.3 Mian Arqam

- Docker deployment
- AWS cloud setup
- DevOps and monitoring

Chapter 8

Challenges Faced

- Real-time synchronization issues
- React Leaflet compatibility problems
- Docker networking configuration
- AWS deployment setup
- MongoDB connection management

Chapter 9

Conclusion

This project demonstrates the practical implementation of cloud computing concepts in disaster management systems. The platform successfully integrates real-time communication, cloud deployment, and scalable architecture into a unified solution.

Future improvements include AI-based prediction systems, mobile application support, and Kubernetes deployment.

References

1. Node.js Documentation
<https://nodejs.org>
2. React Documentation
<https://react.dev>
3. MongoDB Documentation
<https://mongodb.com>
4. AWS Documentation
<https://aws.amazon.com/documentation>
5. Docker Documentation
<https://docs.docker.com>